

Optimal Path Planning in the Presence of Static Obstacles and Uncertain Dynamic Obstacles

Andrew C. Campbell*

The University of Alabama, Tuscaloosa, AL, 35487

In this paper a method for augmenting the Rapidly-exploring Random Tree star (RRT*) algorithm introduced by Karaman & Frazzoli,² such that it can plan in an uncertain dynamic environment, is presented. The innovation is primarily in the development of a grid management system capable of handling multiple obstacle types including uncertain obstacles, such that the probability of a cell being occupied at a certain time can be calculated and passed to any planning system. Notably this grid management system is capable of interfacing with any planner that calculates its own state in relation to time as it plans. This allows for a planning algorithm to avoid future collisions, and avoid regions that a moving obstacle could occupy in the future, all while avoiding static obstacles as any conventional planner does.

I. Introduction

The path and motion planning problem is a fundamental problem in robotics and autonomous systems. There has yet to be an extensive amount of work done on the problem of path planning in the presence of dynamic obstacles, let alone uncertain dynamic obstacles as well. The development then of a system that can manage all these cases at once is necessary to provide more flexibility for the implementation of planners on autonomous systems to run a more robust and reliable system. Presented here is a Grid management system that is capable of managing certain and uncertain static obstacles and certain and uncertain dynamic obstacles. The grid management system is capable of propagating dynamic obstacles into the future a given amount of time with a given time step resolution and can determine the occupancy probability of a cell at a given time. This system can also sample many potential futures for a dynamic obstacle based on the random parameters and boundary conditions of said obstacle. By doing all of this within the grid management system, it can interface with any planner that calculates its time of travel to a node such that it can access the grid as it plans to ensure safe planning in a dynamic uncertain environment. To demonstrate the effectiveness of this system, an RRT* algorithm that follows a set velocity to determine time is paired with a path validation system that checks a bounding box for the size a vehicle for intersections with grid elements at the desired time.

II. Problem Definition

Let $\mathcal{X} \in \mathbb{R}^{n \times 1}$ where $n \in \mathbb{N}$ denote the n -dimensional space and one time dimension configuration space that a vehicle is operating within. $\mathcal{X}_{obs} \in \mathcal{X}$ denotes the set of occupied space and time. $\mathcal{X}_{free} = \mathcal{X} \setminus \mathcal{X}_{obs}$ denotes the set of free space and time. Let the initial state and goal be denoted by $x_{init}, x_{goal} \in \mathcal{X}_{free}$ respectively.

Define the vehicle V as the space confined by a polygon V_{poly} with vertices $\{v_1, v_2, \dots, v_n\}$ in $\mathbb{R}^n$. The position and orientation of the vehicle at any time t can be represented by a transformation $T(x, V)$, where x is a state in $\mathcal{X}$ and $T(x, V)$ is a translation and rotation applied to V . The transformed vehicle $V_{transformed} = T(x, V)$ represents the space aligned with the vehicle's position and orientation in the configuration space $\mathcal{X}$. Note that for path planning to be possible $T(x_{init}, x_{goal}, V) \in \mathcal{X}_{free}$ must be true.

*Graduate Research Assistant, Aerospace Engineering & Mechanics, accampbell4@crimson.ua.edu.

A dynamic obstacle $\mathcal{O}$ can be defined similarly to a vehicle as the space with a polygon $\mathcal{O}_{poly}$ with vertices $\{o_1, o_2, \dots, o_n\}$ in $\mathbb{R}^n$ with dynamics that define some state function $x_{obs}(t)$. Then, $\mathcal{X}_{obs} \supset T(x_{obs}(t), \mathcal{O})$.

An Uncertain dynamic obstacle can mean that the polygon defining the obstacle $\mathcal{O}_{poly}$ is not known exactly and/or the the state function $x_{obs}(t)$ is a function of random variables. In this case we define a new $\mathcal{X}_{uncertain} \in \mathcal{X}$ for spaces that have a nonzero but less than or equal to one probability of occupancy, therefore $\mathcal{X}_{obs} \subset \mathcal{X}_{uncertain}$.

Both static certain and uncertain obstacles can be defined in the same way as dynamic obstacles, but with a state function that is invariant with time.

The goal of a path planner is to find a path $\mathcal{P} = \{x_1, x_2, \dots, x_n\}$ as a list of states defined such that $x_1 = x_{init}$, $x_n = x_{goal}$, and $T(\mathcal{P}, V) \subset \mathcal{X}_{free}$.

III. RRT* Algorithm and Grid Management System

For an RRT* planning algorithm let $\mathcal{T} = \mathcal{T}(\mathcal{V}, E)$ define the tree where $\mathcal{V}$ are the vertices and E are the edges. let $N \in \mathbb{N}$ denote the number of samples that the algorithm will take to plan. Let r define the maximum edge length for a tree. a discrete cost function to some horizon ℓ is conventionally defined generally as:

$$J = c_\ell(x_\ell) + \sum_{t=0}^{\ell-1} c_t(x_t, u_t)$$

where $c_\ell(x_\ell)$ and $c_t(x_t, u_t)$ are local cost functions for the discrete space and x is a state and u is an input. The first term then is the cost at the horizon and the second term is the cost to said horizon. When applied we can define our cost function $c(x_t, u_t)$ in any way we like to tune or guide a planner to minimize this function over $\mathcal{X}$. To follow this structure a planner will have to implement the cost function such that it is summed with all parent nodes to the current node (the horizon) in this case.

Let $\mathcal{G}$ represent the configuration space $\mathcal{X}$ voxelized into cells that contain information on probability of being occupied, location, and cell size, along with methods to manage the space and how obstacles interact with it. The algorithms and system structure are as follows:

Algorithm 1: RRT*

Input: x_{init} , x_{goal} , N
Output: $\mathcal{P}$ from x_{init} to x_{goal}
 $\mathcal{T} \leftarrow \{x_{init}\};$
for $i \leftarrow 1$ **to** N **do**
 $x_{rand} \leftarrow \text{SampleRandomState}();$
 $x_{nearest} \leftarrow \text{NearestNeighbor}(\mathcal{T}, x_{rand});$
 $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand}, r);$
 if $\text{Validator.CollisionFree}(x_{nearest}, x_{new})$ **then**
 $cost_{new} \leftarrow \text{costFunction}(x_{nearest}, x_{new}) + \mathcal{T}.\text{at}(x_{nearest}).\text{cost};$
 $time_{new} \leftarrow \text{timeToTravel}(x_{nearest}, x_{new}) + \mathcal{T}.\text{at}(x_{nearest}).\text{time};$
 $\mathcal{T}.\text{addNode}(x_{nearest}, x_{new}, cost_{new}, time_{new});$
 $X_{near} \leftarrow \text{Near}(\mathcal{T}, x_{new}, r);$
 $\text{Rewire}(x_{new}, X_{near});$
 end
end
while $currentNode \neq x_{init}$ **do**
 $\mathcal{P}.\text{push_front}(currentNode);$
 $currentNode \leftarrow \mathcal{T}.\text{parent}(currentNode);$
end
return $\mathcal{P};$

Algorithm 2: Rewire()

Input: x_{new}, X_{near}
foreach $x_{near} \in X_{near}$ **do**
 $cost_{near} \leftarrow \text{costFunction}(x_{new}, x_{near}) + \mathcal{T}.\text{at}(x_{new}).\text{cost};$
 $time_{near} \leftarrow \text{timeToTravel}(x_{new}, x_{near}) + \mathcal{T}.\text{at}(x_{new}).\text{time};$
 if $cost_{near} < \mathcal{T}.\text{at}(x_{near}).\text{cost}$ **then**
 $\mathcal{T}.\text{at}(x_{near}).\text{parent} \leftarrow x_{new};$
 $\mathcal{T}.\text{at}(x_{near}).\text{cost} \leftarrow cost_{near};$
 $\mathcal{T}.\text{at}(x_{near}).\text{time} \leftarrow time_{near};$
 end
end

Algorithm 3: CostFunction()

Input: x_1, x_2
distance $\leftarrow \text{norm}(x_1 - x_2);$
collisionProbability $\leftarrow \text{Validator.getCollisionProbability}(x_1, x_2);$
return distance + collisionProbability * *PunishmentConst*;

Algorithm 4: Validator

Input: $x_1, x_2, \text{Probability}, \mathcal{V}, \text{time}$
 $\text{PathPolygon} \leftarrow \text{extendPolygon}(T(\{x_1, x_2\}, \mathcal{V}));$
foreach $\mathcal{C} = \mathcal{G}.\text{cell}.\text{at}(\text{time}) \in \mathcal{X}_{\text{uncertain}}$ **do**
 if $\mathcal{C} \cap (\text{PathPolygon})$ **then**
 if $\mathcal{C} \in \mathcal{X}_{\text{obs}}$ **then**
 return false;
 end
 else
 if $\mathcal{C} \cap \mathcal{X}_{\text{uncertain}} \neq \emptyset$ **and** $\mathcal{C}.\text{probability} > \text{Probability}$ **then**
 $\text{Probability} \leftarrow \mathcal{C}.\text{probability};$
 end
 end
 end
end
return true;

Algorithm 5: Grid Management System

Input: $\mathcal{X}$, $\mathcal{X}_{obs}$, $\mathcal{X}_{uncertain}$, $\{\mathcal{O}_1, \mathcal{O}_2, \dots, \mathcal{O}_n\}$

Output: Updated $\mathcal{G}$ with obstacle information

$\mathcal{G} \leftarrow \text{UpdateWithGivenInputs}(\mathcal{X}, \mathcal{X}_{obs}, \mathcal{X}_{uncertain});$

foreach $\mathcal{O} \in \{\mathcal{O}_1, \mathcal{O}_2, \dots, \mathcal{O}_n\}$ **do**

foreach $t \in \text{timeRange}$ **do**

if $\mathcal{O}$ is certain **then**

$x_{obs}(t) \leftarrow \mathcal{O}.\text{state}(t);$

$O_{transformed} \leftarrow T(x_{obs}(t), \mathcal{O}_{poly});$

else

$x_{obs}(t) \leftarrow \mathcal{O}.\text{SampleStates}()(t)$ // generates futures for dynamic Obstacles

$O_{transformed} \leftarrow T(x_{obs}(t), \mathcal{O}_{poly});$

end

end

foreach $\mathcal{C} \in \mathcal{G}.\text{cells}$ **do**

if $\mathcal{C} \cap O_{transformed} \neq \emptyset$ **then**

if $\mathcal{O}$ is certain **then**

$\mathcal{C}.\text{setOccupied}(t);$

end

else

$\mathcal{C}.\text{updateProbability}(t, \mathcal{O}.\text{probability});$

end

end

end

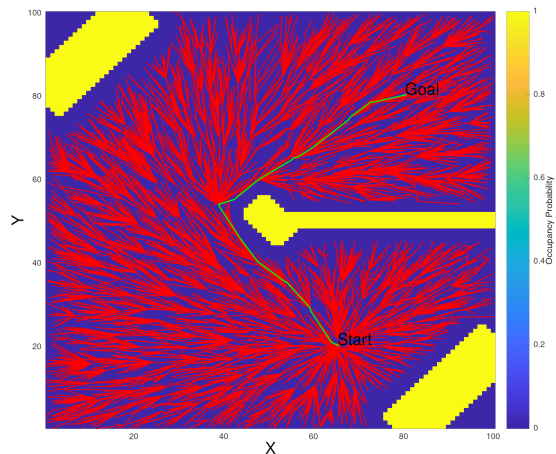
end

end

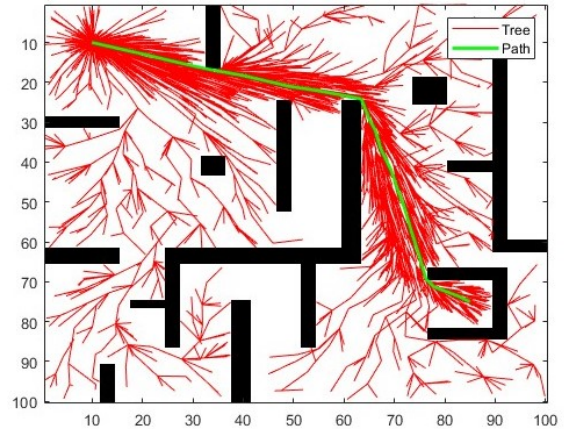
return $\mathcal{G};$

IV. Results

An RRT* algorithm will yield a path that continues to converge towards the optimal path with more iterations. Of interest when using the algorithm layout in **Algorithm 1** several functions can be modified to affect the behavior of the algorithm. the standard implementation of RRT* with a static grid yields a path and search tree resembling arms reaching out from the start point like in **Figure 1a**. However by interchanging processes within the algorithm different behaviors can be achieved for example the implementation in **Figure 1b**. However RRT* algorithms can also be augmented with vehicle dynamics to change the steer function to improve path feasibility. The grid management has been designed to maintain this modularity and further the capabilities for any planner that determines time to travel as well.



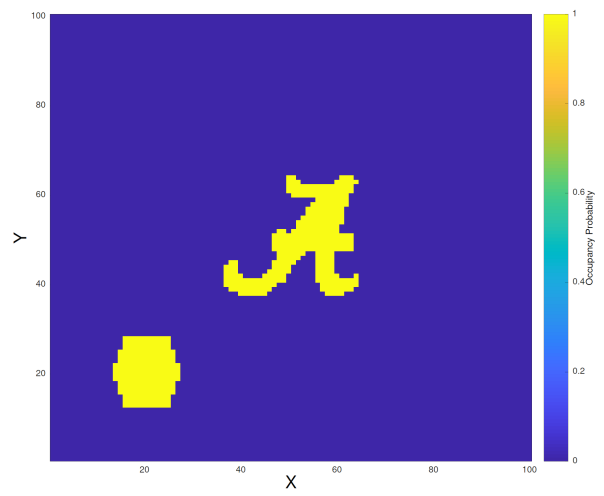
(a) RRT* in Static Grid With Path(Green) and Search Tree(Red)



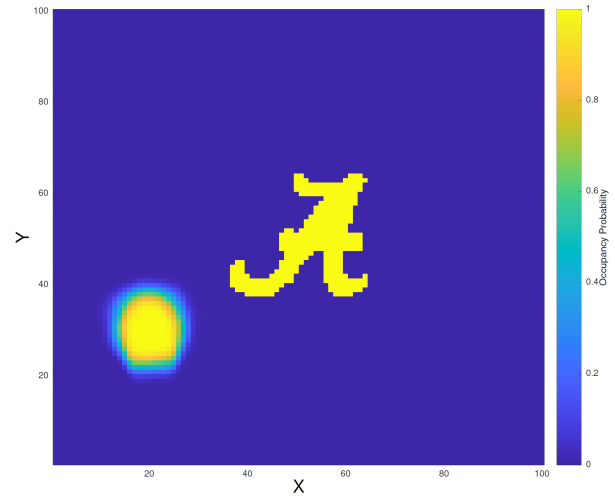
(b) EP-RRT* described by Ding et. al.¹ in a static grid

Figure 1: Example RRT* algorithms

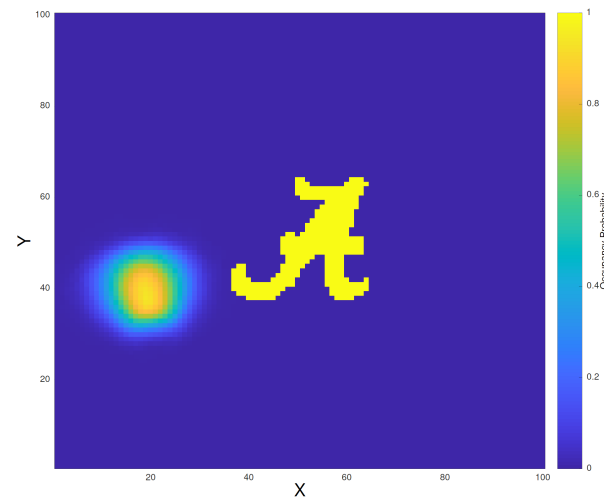
To examine the effectiveness of the proposed algorithms the test environment was set up as a $10m \times 10m \times 1min$ configuration space with a static certain obstacle in the center and an uncertain dynamic obstacle with a starting orientation pointing north traveling at a forward velocity of $0.2 m/s$, with a mean velocity of $0.2 m/s$, velocity variance $0.1 m/s$, mean turn rate of $1^\circ/s$ and turn rate variance of $8.59437^\circ/s$ and using 1000 samples of projected futures.



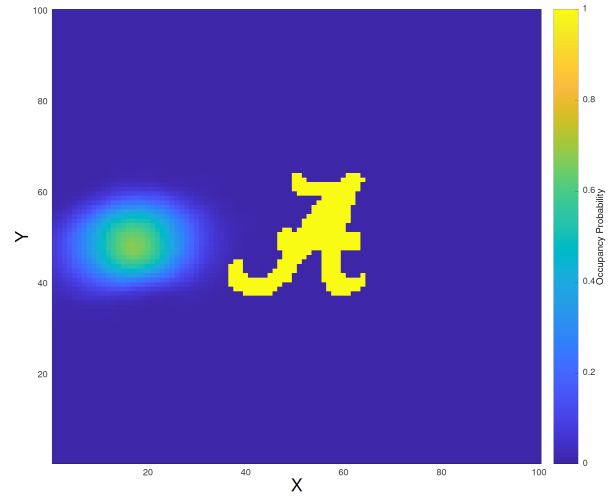
(a) Grid at $time = 0$



(b) Grid at $time = 5$



(c) Grid at $time = 10$



(d) Grid at $time = 15$

Figure 2: Grid cell Probability as time Progresses

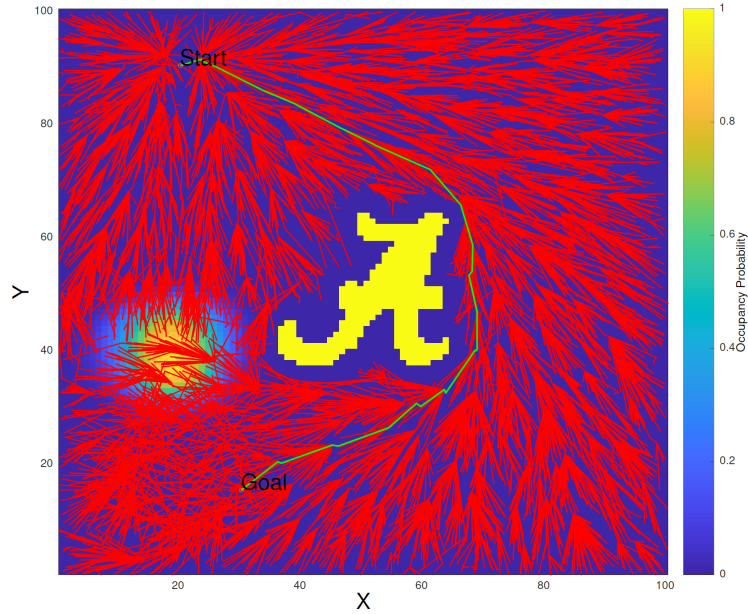


Figure 3: Solved Path(Green) and Search Tree(Red) Overlaid on the Grid at $time = 10$

Note how the path is not the shortest distance but rather it goes the safe way around the center to avoid a collision. The tree's behavior is also far more chaotic in regions where the obstacle may be in the future as the cost function and thus the rewire step punished potential collisions and all time points for the tree are truncated onto the xy plane for easier visualization of the behavior of the planner.

V. Conclusion

It is shown that using this grid system, a safe path, in a dynamic environment can be determined even when the future is uncertain. Among the need for more testing to quantify robustness, runtime, efficiency, and comparison with other approaches, the current work can use further development for the implementation of more complex dynamics for both obstacles and vehicles, requiring the creation of a dynamics class to manage the behavior of these items on its own. Optimizations of the sampling system are necessary as well and should likely include a polynomial chaos expansion of the solved dynamic equations to reduce the number of required samples to converge on the predicted future probability field.

References

- ¹Jun Ding, Yinxuan Zhou, Xia Huang, Kun Song, Shiqing Lu, and Lusheng Wang. An improved rrt* algorithm for robot path planning based on path expansion heuristic sampling. *Journal of Computational Science*, 67:101937, 2023.
- ²Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, 30(7):846–894, 2011.